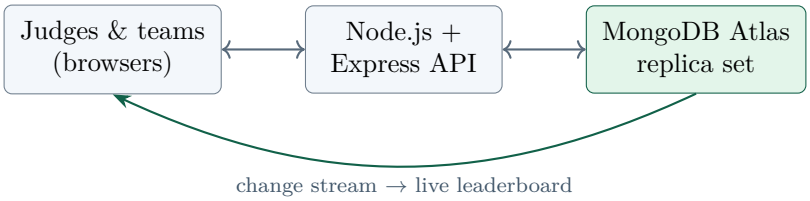
ScoreStream

A Real-Time Event Judging System Built on MongoDB

Project Report — NoSQL Databases



Prepared by: _____

Roll number(s): _____

Guide: _____

Live site: https://scorestream-rfqx.onrender.com

Stack: MongoDB Atlas 8.0 (3-node replica set) · Node.js MongoDB driver v7 · Express 5 · HTML/JS

Abstract

ScoreStream runs judged competitions such as hackathons and project expos. Organisers create events with a weighted rubric, invite judges and teams by their permanent codes, judges score teams one at a time, and every team sees its rank change live. When the event ends, an AI model turns the judges' short notes into written feedback.

This report concentrates on **how MongoDB is used**: where data lives in the cluster and how the cluster keeps copies in sync, how documents are modelled, how unique judge and team codes are generated and why they can never clash, which indexes exist and what they prove, every aggregation pipeline (`$match`, `$group`, `$lookup`, `$setWindowFields`, `$merge`, ...), how Change Streams push live updates, and how slide files are stored with GridFS. Every number in the measurement tables was produced by running the real queries against the real Atlas cluster.

Contents

1 Why MongoDB for this project **3**

2 MongoDB architecture: cluster, database, documents **3**

2.1 The Atlas cluster is a replica set . . . 3

2.2 How the server connects . . . 4

2.3 Database → collections → documents . . . 4

3 Data model **4**

3.1 The collections . . . 4

3.2 How the collections reference each other . . . 5

3.3 Embedding vs referencing — the main design decision . . . 5

3.4 Example documents . . . 6

4 Schema validation with $jsonSchema **6**

5 Judge and team codes: generation and uniqueness **7**

5.1 How a code is generated . . . 7

5.2 How we make sure no two judges get the same code . . . 7

5.3 No duplicate invitations, no judge added twice . . . 8

6 Indexes **9**

6.1 All indexes . . . 9

6.2 Proof: COLLSCAN vs IXSCAN . . . 9

7 CRUD operations used **10**

8 Aggregation pipelines **10**

8.1 The leaderboard pipeline (`src/services/leaderboard.js`) 10
8.2 Per-criterion averages for a team (`$objectToArray`) 12
8.3 Counting without copying: `$lookup` with a sub-pipeline 12
8.4 Other pipelines in the app 12

9 Real-time synchronisation: Change Streams **13**
9.1 From a judge's click to every screen 13
9.2 The change stream (`src/services/changeStream.js`) 13
9.3 What else is synchronised live 14
9.4 Consistency guarantees 14

10 File storage with GridFS **14**

11 Security and data protection **15**

12 Likely viva questions — short answers **15**

A Commands **17**

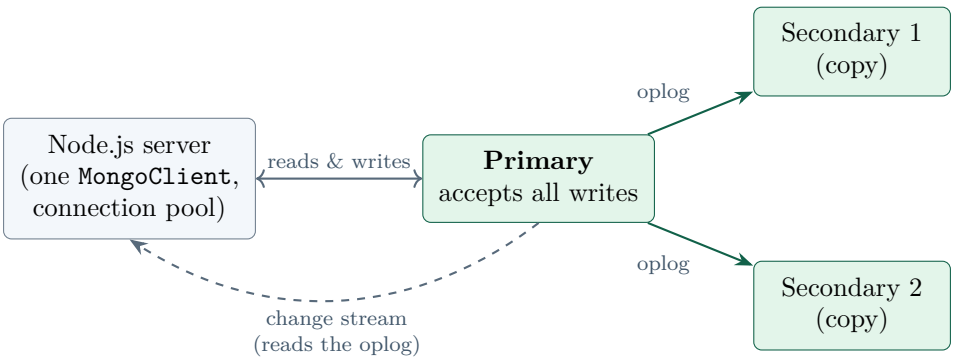
1 Why MongoDB for this project

- **Each event has its own rubric.** One hackathon scores *Innovation, Design, Execution*; another might score *Idea, Pitch*. A score document simply stores `criteria: { innovation: 8, design: 7 }` with whatever keys that event uses. In SQL this would need an extra criteria table and joins, or a schema change per event.
- **Data is read the way it is stored.** An event and its rubric are always needed together, so the rubric is *embedded* inside the event document: one read, no join.
- **The database does the maths.** Averages, judge counts and competition ranks are produced by one *aggregation pipeline* running inside MongoDB, and saved with `$merge`.
- **Real-time out of the box.** *Change Streams* tell the server the moment a score changes. There is no polling loop.
- **Files in the same database.** *GridFS* stores the teams' slides, so no separate file server is needed and files survive server redeploys.

2 MongoDB architecture: cluster, database, documents

2.1 The Atlas cluster is a replica set

The data lives in a **MongoDB Atlas** cluster. We checked its shape with the `hello` command: it is a **replica set of 3 members** running **MongoDB 8.0.34**.



- **Primary**: every insert, update and delete goes here first.
- **Oplog** (operations log): the primary records each write in a special capped collection. The two **secondaries** copy the oplog and replay it, so all three hold the same data.
- **Automatic failover**: if the primary goes down, the secondaries elect a new primary within seconds and the driver reconnects by itself. The app keeps working.
- **Why we need a replica set**: Change Streams (Section 9) are built on the oplog, so they only work on a replica set. Atlas clusters are always replica sets.

2.2 How the server connects

One `MongoClient` is created at start-up and reused by every request (`src/db.js`). The client keeps a **connection pool** inside it, so requests do not pay the cost of opening a new connection.

```
client = new MongoClient(config.mongodbUri);   // mongodb+srv://... from .env
await client.connect();
db = client.db(config.dbName);                 // database "judging_system"
await db.command({ ping: 1 });                 // proves the cluster answers
```

The connection string uses the `mongodb+srv://` scheme: the driver looks up DNS records to find all three replica-set members automatically. The string (with the password) lives only in the server's `.env` file, which is never committed to Git.

2.3 Database → collections → documents

- **Database**: `judging_system`. Tests use a separate `judging_system_test` database that is dropped after every run, so tests never touch real data.
- **Collection**: a group of documents, like a table, but without a fixed set of columns.
- **Document**: a JSON-like record stored as **BSON** (Binary JSON). BSON adds real types JSON lacks: `ObjectId`, `Date`, 64-bit integers and binary data. A score document averages **193 bytes** (measured with `collStats`).
- **_id / ObjectId**: every document has a unique `_id`. By default it is an `ObjectId`: 12 bytes = a 4-byte **timestamp** + a 5-byte random value + a 3-byte counter. It is created by the driver, is unique without asking the server, and roughly sorts by creation time.
- **Storage engine**: **WiredTiger**, which stores documents compressed on disk and keeps the most used data and indexes in memory.

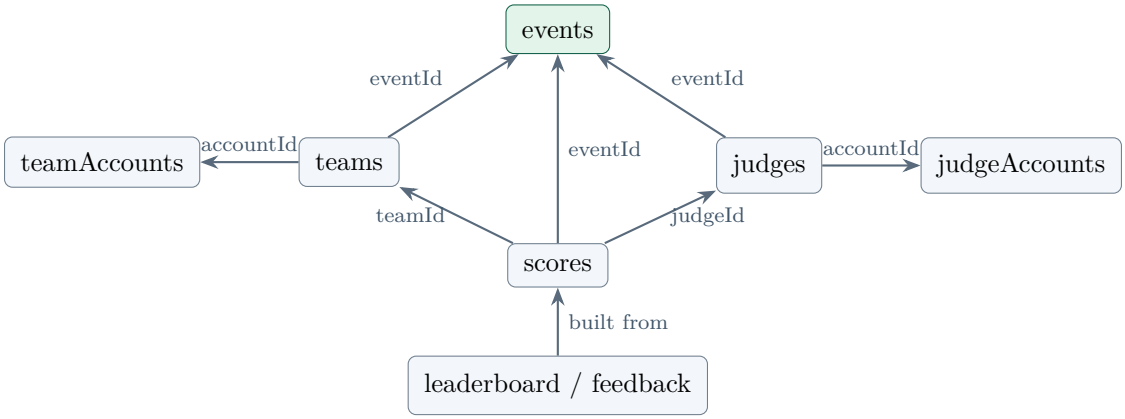
3 Data model

3.1 The collections

Collection	Holds	Key fields
`events`	One competition, with its rubric *embedded*	name, status (draft/live/closed), rubric[], ownerId
`admins`	Organiser accounts	username, usernameLower, passwordHash
`judgeAccounts`	A registered judge (permanent)	name, **code** (JG…), passwordHash
`teamAccounts`	A registered team (permanent)	name, members[], **code** (TM…), passwordHash
`judgeInvitations`	Admin asks a judge to join an event	eventId, accountId, status, invitedBy

Collection	Holds	Key fields
`invitations`	Admin asks a team to join an event	eventId, accountId, status, invitedBy
`judges`	A judge's *seat* in one event	eventId, accountId, name, joinedAt
`teams`	A team's *place* in one event	eventId, accountId, name, members[]
`scores`	One judge's marks for one team	eventId, teamId, judgeId, criteria{}, total, comment
`leaderboard`	Ranked results (written only by `$merge`)	eventId, teamId, avgScore, judgesCount, rank
`feedback`	AI feedback per team	summary, strengths[], improvements[], sentiment
`submissions`	A team's GitHub link + slides info	eventId, teamId, githubUrl, slides{fileId,...}
`slides.files` / `slides.chunks`	**GridFS** file storage	the uploaded PDF/PPT bytes in 255 KB chunks

3.2 How the collections reference each other



3.3 Embedding vs referencing — the main design decision

Choice	Why
Rubric **embedded** in the event	Small (3–6 items), always read with the event, never queried on its own. One read gives everything.
Scores **referenced** (own collection)	Written very often by many judges at once, queried on their own (per event, per team), and the count grows with teams × judges. Embedding them inside a team would create one hot, ever-growing document that every judge writes to.
Accounts vs. seats	A team or judge *account* is permanent and can join many events; the `teams`/`judges` document is its seat in one event. Scores point at the seat, so removing a judge from one event never touches their other events.

3.4 Example documents

An **event** with its embedded rubric:

```
{ "_id": ObjectId("6ade3fca77d4420e1b815c02"),
  "name": "Spring Hackathon", "status": "live",
  "rubric": [ { "key": "innovation", "label": "Innovation", "max": 10, "weight": 0.3 },
              { "key": "design",     "label": "Design",     "max": 10, "weight": 0.4 },
              { "key": "execution",  "label": "Execution",  "max": 10, "weight": 0.3 } ],
  "ownerId": ObjectId("..."), "createdAt": ISODate("2026-10-25T17:40:00Z") }
```

A **score** — `criteria` keys follow the event's rubric:

```
{ "eventId": ObjectId("..."), "teamId": ObjectId("..."), "judgeId": ObjectId("..."),
  "criteria": { "innovation": 10, "design": 6.5, "execution": 5 },
  "total": 7.1,
  "comment": "Ui is nice but laggy, the slides could have been better",
  "submittedAt": ISODate("2026-10-25T17:52:11Z") }
```

Scoring formula (computed on the server, never trusted from the browser):

$$total \;=\; 10 \times \sum_{i} \frac{mark_i}{max_i} \times weight_i \qquad \text{e.g. } 10 \times (0.3 \cdot \tfrac{10}{10} + 0.4 \cdot \tfrac{6.5}{10} + 0.3 \cdot \tfrac{5}{10}) = 7.1$$

A **judge account**:

```
{ "_id": ObjectId("..."), "name": "Prof. Rao", "code": "JG3H8M2P",
  "passwordHash": "9f2c...e1:5ab0...77",          // scrypt "salt:hash", never the password
  "createdAt": ISODate("2026-10-25T17:30:00Z") }
```

4 Schema validation with $jsonSchema

MongoDB is schemaless by default. We still want the *database itself* to reject malformed data, even if a bug in the Node code lets something through. `scripts/setup-db.js` attaches a **`$jsonSchema` validator** to all 12 collections with `validationLevel: "strict"` (check every insert and update) and `validationAction: "error"` (reject, don't just warn).

```
scores: { $jsonSchema: {
  bsonType: 'object',
  required: ['eventId', 'teamId', 'judgeId', 'criteria', 'total', 'submittedAt'],
  properties: {
    eventId:  { bsonType: 'objectId' },
    criteria: { bsonType: 'object', additionalProperties: { bsonType: 'number' } },
    total:    { bsonType: 'number', minimum: 0, maximum: 10 },
    comment:  { bsonType: 'string' } } } }

judgeAccounts: { $jsonSchema: { ... properties: {
    code:         { bsonType: 'string', pattern: '^JG[A-Z0-9]{6}$' },
    passwordHash: { bsonType: 'string', pattern: '^[a-f0-9]{32}:[a-f0-9]{128}$' } } } }
```

- `criteria` accepts *any* keys (each event's rubric differs) but *only numbers* as values.
- The `passwordHash` pattern means a plain-text password can never be stored by mistake.

- **Proof**: inserting `criteria: { innovation: "ten" }, total: 99` is refused by the server with `code 121: Document failed validation`.
- Rules a schema cannot express (*weights add up to 1*, *criteria keys match this event's rubric*) are checked in Node before writing.

5 Judge and team codes: generation and uniqueness

5.1 How a code is generated

Every judge and team gets a permanent code: `JG` or `TM` followed by 6 characters, e.g. `JG3H8M2P`. The code is what the organiser types to invite them, and what they use to sign in (with their password). From `src/middleware/auth.js`:

```
// No 0/O or 1/I, so codes are easy to read out loud.
const CODE_LETTERS = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';   // 32 characters

function generateAccountCode(prefix) {
  const bytes = crypto.randomBytes(6);        // cryptographically secure random bytes
  let code = prefix;
  for (const byte of bytes) code += CODE_LETTERS[byte % CODE_LETTERS.length];
  return code;
}
const generateJudgeCode = () => generateAccountCode('JG');
```

- `crypto.randomBytes` comes from the operating system's secure generator (not `Math.random`), so codes cannot be predicted.
- 32 symbols, 6 positions: $32^6 = 1{,}073{,}741{,}824$ possible codes per prefix.
- A byte has 256 values and $256 = 8 \times 32$, so `byte % 32` picks every symbol with exactly equal probability (no *modulo bias*).

5.2 How we make sure no two judges get the same code

Random codes *can* collide in theory. The guarantee comes from MongoDB, not from luck:

```
// scripts/setup-db.js -- a UNIQUE index on the code
db.judgeAccounts.createIndex({ code: 1 }, { unique: true, name: 'unique_code' })
db.teamAccounts.createIndex({ code: 1 },  { unique: true, name: 'unique_code' })
```

With a unique index, the *database* refuses a second document with an existing code and returns **error 11000 (duplicate key)**. Registration simply tries again with a fresh code (`src/routes/auth.js`):

```
async function insertWithCode(collection, account, makeCode) {
  for (let attempt = 1; ; attempt++) {
    account.code = makeCode();
    try {
      const result = await collection.insertOne(account);
      return result.insertedId;                       // code was free -> done
    } catch (err) {
      if (err.code !== 11000 || attempt >= 5) throw err; // 11000 = code already taken
      delete account._id;                             // retry with a new random code
```

```
    }
  }
}
```

Why not "check if the code exists, then insert"? Two registrations at the same instant could both check, both see the code is free, and both insert it — a *race condition*. A unique index is checked *atomically inside the insert*, so exactly one of them can win. The chance of even one retry is tiny: with 10,000 judges already registered, a new random code clashes with probability $10{,}000/32^6 \approx 0.0009\%$.

Signing in uses the same index to find the account instantly, then checks the password:

```
const account = await db.collection('judgeAccounts').findOne({ code: code.trim().toUpperCase()
    });
if (!account || !checkPassword(password, account.passwordHash)) throw httpError(401, 'Wrong
    judge code or password');
```

Passwords are stored only as **scrypt** hashes with a random 16-byte salt, compared with `crypto.timingSafeEqual`.

5.3 No duplicate invitations, no judge added twice

Joining an event is a two-step handshake, and both steps are protected by indexes:

1. The admin enters the judge's code → a `judgeInvitations` document with `status: "pending"`.
2. The judge accepts on their dashboard → a `judges` document (their seat) is inserted.

```
// At most ONE pending invitation per judge per event. PARTIAL index: only documents
// with status 'pending' are in it, so old accepted/declined invitations never clash.
db.judgeInvitations.createIndex({ eventId: 1, accountId: 1 },
  { unique: true, partialFilterExpression: { status: 'pending' } })

// A judge account can hold at most one seat per event.
db.judges.createIndex({ eventId: 1, accountId: 1 }, { unique: true })
```

Teams use exactly the same pattern (`invitations` and `teams`). Inviting the same judge twice, or clicking *Accept* twice in two tabs, ends in error 11000 instead of a duplicate.

Viva point

"What stops a judge scoring the same team twice?" — The unique compound index `{eventId, teamId, judgeId}` on `scores`. We verified it: inserting a copy of an existing score returns `E11000 duplicate key error ...index: unique_event_team_judge`. Editing a score uses `findOneAndUpdate` on the existing document instead.

6 Indexes

6.1 All indexes

Collection	Index	Purpose
`scores`	`{eventId, teamId, judgeId}` **unique**	One score per judge per team (no double scoring)
`scores`	`{eventId}`	Fast "all scores of this event" — first stage of the leaderboard pipeline
`leaderboard`	`{eventId, teamId}` **unique**	Required by `$merge on: [eventId, teamId]`
`feedback`	`{eventId, teamId}` **unique**	One AI review per team; regenerate = upsert
`judgeAccounts`	`{code}` **unique**	Unique judge codes + fast login
`teamAccounts`	`{code}` **unique**	Unique team codes + fast login
`judges`	`{eventId, accountId}` **unique**	A judge sits in an event once
`judges`	`{accountId}`	"Which events does this judge judge?"
`teams`	`{eventId, accountId}` **unique, partial**	A team joins an event once
`judgeInvitations`	`{eventId, accountId}` **unique, partial**	One *pending* invite per judge per event
`invitations`	`{eventId, accountId}` **unique, partial**	One *pending* invite per team per event
`submissions`	`{eventId, teamId}` **unique**	One submission per team per event
`admins`	`{usernameLower}` **unique**	"Asha" and "asha" can't both register
`events`	`{ownerId, createdAt: -1}`	An organiser's events, newest first

Index types used: *single-field*, *compound* (several fields, order matters), *unique*, and *partial* (only documents matching a filter are indexed).

6.2 Proof: COLLSCAN vs IXSCAN

`scripts/explain.js` runs `.explain("executionStats")` on our two most important queries. We ran it on a copy of the schema with **500 scores** (5 events × 20 teams × 5 judges), once with the indexes and once after dropping them:

Query	Index	Plan (stages)	Returned	Keys read	Docs read
`find({eventId})`	none	`COLLSCAN`	100	0	**500**
`find({eventId})`	yes	`IXSCAN` → `FETCH`	100	100	**100**
Leaderboard pipeline	none	`COLLSCAN` → `GROUP`	20	0	**500**
Leaderboard pipeline	yes	`IXSCAN` → `FETCH` → `GROUP`	20	100	**100**

In both "yes" rows MongoDB chose the `eventId_1` index.

Without the index MongoDB reads *every* score in the collection (all events) to find one event's 100. With it, it jumps straight to those 100. The difference grows with the data: with 100 events it would be 100 documents read instead of 10,000. To reproduce: `npm run explain -- --drop-indexes`, then `npm run setup-db`, then `npm run explain`.

7 CRUD operations used

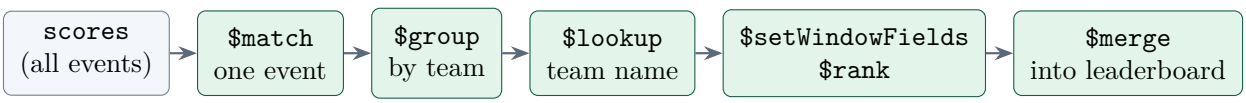
Operation	Where it is used
`insertOne`	Register an account, create an event, invitation, seat, submit a score
`insertMany`	Seeding test data
`findOne`	Log in by code; load an event; "is this judge in this event?" (`findJudgeSeat`)
`find + projection + sort`	Leaderboard rows sorted by rank; never sending `passwordHash` (`projection: {passwordHash: 0}`)
`countDocuments`	Number of teams in an event
`updateOne + $set`	Answer an invitation (`status: "accepted"`, `respondedAt`)
`findOneAndUpdate`	Edit a score (filter includes `judgeId`, so a judge can only edit their own); change event status; `returnDocument: "after"`
`upsert`	Save a GitHub link / feedback: update if it exists, insert if not (`replaceOne(..., {upsert: true})`)
`$unset`	Remove a GitHub link or slides from a submission
`deleteMany`	Cascade delete: removing an event deletes its teams, judges, scores, leaderboard, feedback, invitations (MongoDB has no `ON DELETE CASCADE`)

8 Aggregation pipelines

An aggregation pipeline is a list of **stages**. Documents flow through them in order and each stage transforms the stream: filter, group, join, sort, rank, reshape, write. The work runs *inside the database*, next to the data, instead of loading everything into Node.

8.1 The leaderboard pipeline (`src/services/leaderboard.js`)

This is the heart of the project: it turns raw scores into a ranked leaderboard.




```
[
  // 1. $match FIRST: only this event's scores. Uses the { eventId: 1 } index (IXSCAN),
  //    so every later stage works on fewer documents.
  { $match: { eventId } },
```

```
  // 2. $group: one output document per team.
  { $group: { _id: '$teamId',
              avgScore:    { $avg: '$total' },   // average of every judge's total
              judgesCount: { $sum: 1 } } },       // +1 per score = number of judges

  // Round BEFORE ranking, so two teams showing 7.25 share a rank.
  { $set: { avgScore: { $round: ['$avgScore', 2] } } },

  // 3. $lookup = join with teams to get the name (scores store only teamId).
  { $lookup: { from: 'teams', localField: '_id', foreignField: '_id', as: 'team' } },
  { $unwind: '$team' },        // [ {team} ] -> {team}; drops scores of deleted teams

  // 4. Window function: rank by average, highest first. Ties share a rank: 1, 2, 2, 4.
  { $setWindowFields: { sortBy: { avgScore: -1 }, output: { rank: { $rank: {} } } } },

  // Shape each row like a leaderboard document.
  { $project: { _id: 0, eventId: { $literal: eventId }, teamId: '$_id', teamName: '$team.name',
                avgScore: 1, judgesCount: 1, rank: 1, updatedAt: { $literal: now } } },

  // 5. $merge: write the rows into the leaderboard collection.
  { $merge: { into: 'leaderboard', on: ['eventId', 'teamId'],
              whenMatched: 'replace', whenNotMatched: 'insert' } }
]
```

What each stage does

Stage	Meaning
`$match`	Filter, like SQL `WHERE`. Placed first so it can use an index.
`$group`	Like SQL `GROUP BY`. `_id` is the grouping key; accumulators `$avg`, `$sum` compute per group.
`$set / $round`	Add or overwrite a field; round to 2 decimals.
`$lookup`	Left outer join with another collection; the matches arrive as an array.
`$unwind`	One output document per array element (turns `[team]` into `team`).
`$setWindowFields`	Window function (MongoDB 5.0+): computes a value using neighbouring documents — here `$rank` over the sorted list.
`$project`	Choose, rename and compute output fields (like SQL `SELECT`).
`$merge`	Write results into a collection, updating matching rows (needs a unique index on the `on` fields).

Worked example

Real output from the 500-score test data (event 3, 20 teams, 5 judges each):

rank	teamName	avgScore	judgesCount
1	Team 3-6	7.9	5
2	Team 3-12	7.3	5
3	Team 3-16	7.1	5
4	Team 3-18	6.7	5

Viva point

`$merge` vs `$out`: `$out` replaces the *whole* target collection, which would wipe every other event's leaderboard. `$merge` updates only the matching rows (`on: [eventId, teamId]`). Rows that `$merge` did not touch (a team whose scores were all deleted) are removed afterwards with `deleteMany({ eventId, updatedAt: { $lt: now } })`.

8.2 Per-criterion averages for a team (`$objectToArray`)

The team page shows "Design 6.5/10, Innovation 10/10…". Because `criteria` is an object with event-specific keys, we first turn it into an array of key/value pairs:

```
[ { $match:   { eventId, teamId } },
  // { design: 6.5, innovation: 10 }  ->  [ {k:'design', v:6.5}, {k:'innovation', v:10} ]
  { $project: { pairs: { $objectToArray: '$criteria' } } },
  // one document per key/value pair
  { $unwind:  '$pairs' },
  // average mark per criterion
  { $group:   { _id: '$pairs.k', avg: { $avg: '$pairs.v' } } } ]
```

8.3 Counting without copying: `$lookup` with a sub-pipeline

The admin's event list shows how many teams, judges and scores each event has. A `$lookup` with its own `pipeline` that ends in `$count` returns just a number, without copying hundreds of documents:

```
{ $lookup: { from: 'scores',
             let: { id: '$_id' },
             pipeline: [ { $match: { $expr: { $eq: ['$eventId', '$$id'] } } },
                         { $count: 'n' } ],
             as: 'scores' } },
{ $set: { scoreCount: { $ifNull: [ { $first: '$scores.n' }, 0 ] } } }
```

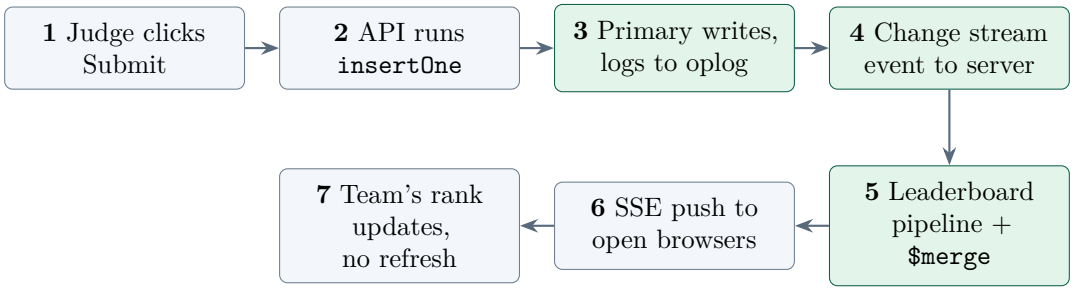
`$$id` refers to the variable defined in `let`; `$expr` lets a `$match` compare two fields.

8.4 Other pipelines in the app

Pipeline	Stages
Judge dashboard	`$match` seats by accountId → `$lookup` events → `$lookup`+`$count` teams and own scores → `$project` progress
Judges list (admin)	`$match` event → `$lookup` judgeAccounts (code) → `$lookup`+`$count` scores per judge
Team dashboard	`$match` by accountId → `$lookup` events → `$lookup` leaderboard with `let` + `$expr` on two fields → `$project` rank
Judges' scores for a team	`$match` → `$lookup` judges (names) → `$project` with `$ifNull` → `$sort`
Teams for a judge	`$match` event → `$lookup` submissions → `$set` githubUrl/slides with `$first` → `$project` hiding internal ids
Pending invitations	`$match status: "pending"` → `$lookup` events/accounts → `$unwind` → `$sort`

9 Real-time synchronisation: Change Streams

9.1 From a judge's click to every screen



9.2 The change stream (`src/services/changeStream.js`)

```
stream = db.collection('scores').watch(
  // A pipeline that runs INSIDE MongoDB: only these 4 kinds of change are sent to us.
  [ { $match: { operationType: { $in: ['insert', 'update', 'replace', 'delete'] } } } ],
  // For updates, also send the whole updated document (we need its eventId).
  { fullDocument: 'updateLookup' }
);

stream.on('change', (change) => {
  // Handle changes one at a time, in order, so two recomputes never race.
  queue = queue.then(() => handleChange(change));
});
```

- `watch()` opens a cursor on the **oplog**. MongoDB *pushes* each change to the server — the server never polls the database.
- `handleChange` reads `change.fullDocument.eventId` and recomputes *only that event's* leaderboard. A `delete` event carries only the deleted `_id`, so in that case every live event is recomputed.
- The handler writes to `leaderboard`, never to `scores`. Writing to the watched collection would trigger itself again in an endless loop.
- The result is pushed to browsers with **Server-Sent Events** (one long-lived HTTP response per open page; the browser's `EventSource` reconnects by itself). A heartbeat every 25 s keeps proxies from closing idle connections.

9.3 What else is synchronised live

Event	Who updates without refreshing
A score is saved or edited	Team: rank, average, judges' marks. Admin: rankings, judging progress. Public leaderboard.
Admin sets Live / ends the event	Judge: stars unlock or lock. Team: status, submission lock.
A team or judge accepts an invitation	Admin: the team/judge appears in the event's list. Judge: a new team appears.
AI feedback is written	Team: the feedback card fills in.

9.4 Consistency guarantees

- **Single-document atomicity**: every write in MongoDB is atomic per document, so a score is never half-saved.
- **Constraints in the database**: unique indexes and `$jsonSchema` validators are enforced by the server for *every* client, even if two requests arrive at the same millisecond.
- **Durability**: writes are acknowledged by a majority of the replica set (the Atlas default write concern `w: "majority"`), so an acknowledged score survives the loss of the primary.
- **Ordered processing**: change events are handled one after another through a promise queue.

10 File storage with GridFS

Teams upload slides (PDF, PowerPoint, Word, Excel, images, up to 20 MB). A single MongoDB document is limited to 16 MB, so files are stored with **GridFS**, MongoDB's built-in file system:

- `slides.chunks`: the file split into **255 KB chunks** (a 2.9 MB PDF became 12 chunks).
- `slides.files`: one document per file: name, length, upload date and our metadata (eventId, teamId, content type).
- `submissions`: the team's document points at the file through `slides.fileId`.

```
const bucket = new GridFSBucket(db, { bucketName: 'slides' });
const upload = bucket.openUploadStream(filename, { metadata: { eventId, teamId, contentType } })
    ;
// The request body is STREAMED into GridFS chunk by chunk (never the whole file in memory),
// through a Transform that aborts once 20 MB is exceeded.
await pipeline(req, sizeLimit, upload);
```

Files live in the database, so they are replicated to all three members and survive server redeploys (the hosting platform's own disk is wiped on every deploy). Replacing slides deletes the old file only *after* the new one is saved; deleting an event deletes its files too.

11 Security and data protection

- Passwords: **scrypt** hash with a random salt; the validator pattern forbids plain text.
- Queries never return secrets: `projection: { passwordHash: 0 }`.
- Every request is authorised against the database: a judge can score only teams of an event where they hold a seat (`findOne({ eventId, accountId })`), and only while the event is `live`; an organiser can manage only events whose `ownerId` is theirs.
- The connection string and API keys live in `.env`, which is excluded from Git.
- Atlas encrypts connections (TLS) and data at rest.

12 Likely viva questions — short answers

Question	Answer
Why MongoDB and not MySQL?	Each event has a different rubric, so score documents have flexible `criteria` keys. The rubric is embedded (one read). Aggregation pipelines rank teams inside the database, Change Streams give real-time updates, GridFS stores files — all in one system.
What is a document? A collection?	A BSON (binary JSON) record with a unique `_id`; a collection is a group of documents, like a table without fixed columns.
What is BSON?	Binary JSON: faster to scan and adds types JSON lacks (ObjectId, Date, int64, binary).
What is an ObjectId?	12 bytes: timestamp + random value + counter. Unique without a central counter, roughly time-ordered.
What is a cluster / replica set?	Three servers holding the same data. One primary takes writes; two secondaries copy its oplog. If the primary fails, a new one is elected automatically.
What is the oplog?	A capped collection on the primary that records every write in order. Secondaries replay it; change streams read it.
Why does the app need a replica set?	Change Streams read the oplog, which exists only on replica sets.
How are judge codes generated?	`crypto.randomBytes(6)` mapped onto 32 easy-to-read characters, prefixed `JG`: $32^6 \approx 1.07$ billion combinations.
How do you guarantee no two judges share a code?	Unique index on `judgeAccounts.code`. A clash makes `insertOne` fail with error 11000 and we retry with a new code. The check is atomic inside the database, so there is no race.
How do you stop double scoring?	Unique compound index `{eventId, teamId, judgeId}` on scores (error 11000 on a duplicate).

Question	Answer
What is a partial index and where?	An index over only the documents matching a filter. Invitations: unique only where `status: "pending"`, so a team can be re-invited after declining.
Why put `$match` first?	It filters early and can use an index (IXSCAN); later stages then handle fewer documents.
`$group` vs SQL GROUP BY?	Same idea: `_id` is the key, accumulators like `$avg`, `$sum` aggregate each group.
What does `$lookup` do?	A left outer join with another collection; matches come back as an array (we use `$unwind` or `$first`).
How is rank computed?	`$setWindowFields` with `$rank` sorted by average descending; ties share a rank (1, 2, 2, 4).
`$merge` vs `$out`?	`$out` replaces the whole collection; `$merge` upserts matching rows only, so other events' leaderboards are untouched.
How does the leaderboard update live?	Score write → oplog → change stream → pipeline + `$merge` → Server-Sent Events to browsers.
Why can't the change-stream handler write to scores?	It would trigger a new change event and loop forever; it writes to `leaderboard` only.
What is `fullDocument: "updateLookup"`?	By default an update event contains only the changed fields; this asks MongoDB to include the whole updated document (we need its eventId).
How did you prove the indexes help?	`explain("executionStats")`: without the index COLLSCAN read 500 documents; with it IXSCAN read 100 to return the same 100.
How are files stored?	GridFS: 255 KB chunks in `slides.chunks`, metadata in `slides.files`; needed because a document is limited to 16 MB.
How do you validate data in the database?	`$jsonSchema` validators on all 12 collections, strict level, error action (verified: code 121 *Document failed validation*).
Is it consistent if the primary fails?	Writes are majority-acknowledged, so acknowledged scores survive failover; the driver reconnects to the new primary automatically.
Why no multi-document transactions?	Every critical operation touches one document (atomic), and invariants are enforced by unique indexes; cascade deletes are idempotent and can simply be re-run.
How would it scale further?	Shard large collections by `eventId`: all of one event's scores sit on one shard, so the leaderboard pipeline stays on a single shard.
How is deleting an event handled without foreign keys?	The API deletes related documents in each collection with `deleteMany({ eventId })` and removes the GridFS files.

A Commands

Command	What it does
`npm run setup-db`	Creates all collections with `$jsonSchema` validators and all indexes (safe to re-run).
`npm run explain`	Prints the query plans (IXSCAN/COLLSCAN, keys and documents examined).
`npm run explain -- --drop-indexes`	Drops the score indexes first to show a COLLSCAN (run `setup-db` afterwards).
`npm test`	13 API tests on a separate test database (double scoring, duplicates, invitations, uploads, . . .).
`npm start`	Connects to Atlas, starts the change stream and the web server.